INF201 – Deliverable 3: Mini-RPG battle simulator

Authors: Oskar Rändel Gabrielsen and Martin Hansen **Repository:** https://github.com/MartinHOskarR/inf201-deliverable-3

This notebook presents the classes in the project and runs simulated battles between them.

Module	Classes
`character.py`	`Character`, `Warrior`, `Mage`
`dice.py`	`Dice`
`items.py`	`Weapon`
`battle.py`	`Battle`

Setup

`autoreload` makes the notebook pick up changes in the `.py` modules without restarting the kernel.

In [1]:
```
%load_ext autoreload
%autoreload 2
```

In [2]:
```
import random

from character import Character, Warrior, Mage
from dice import Dice
from items import Weapon
from battle import Battle
```

Several classes use randomness (dice rolls). We call `random.seed(...)` at the start of each cell that rolls dice, so the results are the same every time the notebook is run.

Part 1: Presentation of the classes

`Dice` (`dice.py`)

Models an n-sided dice (n ≥ 2, default 6).

Attributes	`sides` – number of sides
Methods	`roll()` – returns a random integer from 1 to `sides`
	`__call__()` – lets the object itself be called, so `d20()` is the same as `d20.roll()`
	`__repr__()` – returns e.g. `Dice(sides=20)`

In [3]:
```
random.seed(1)
d6 = Dice()
d20 = Dice(sides=20)
print(d6, d20)
print("d20.roll():", d20.roll())
print("d20():     ", d20())

# Roll 1000 times to check that the full range 1–20 is reachable
rolls = [d20() for _ in range(1000)]
print("min:", min(rolls), "max:", max(rolls))
```

```
Dice(sides=6) Dice(sides=20)
d20.roll(): 5
d20():      19
min: 1 max: 20
```

Creating a dice with fewer than 2 sides raises a `ValueError`:

In [4]:
```
try:
    Dice(sides=1)
except ValueError as error:
    print("ValueError:", error)
```

```
ValueError: Dice must have at least 2 sides.
```

`Weapon` (`items.py`)

A weapon uses a `Dice` object to decide its damage (object composition: a weapon *has a* dice).

Attributes	`name`, `damage_dice` (a `Dice`)
Methods	`roll_damage()` – rolls the weapon's dice and returns the damage
	`__repr__()`

In [5]:
```
random.seed(2)
sword = Weapon("Sword", Dice(8))
print(sword)
print("Five sword hits:", [sword.roll_damage() for _ in range(5)])
```

```
Weapon(name=Sword, damage_dice=Dice(sides=8))
Five sword hits: [1, 2, 2, 6, 3]
```

`Character` (`character.py`)

The base class for all characters.

Attributes	`name`, `hp_max`, `damage` (base damage), `weapon` (optional, default `None`), `_hp` (protected)
Properties	`hp` – read-only access to `_hp`
	`is_alive` – read-only, `True` if `_hp > 0`
Methods	`take_damage(damage)` – lowers the character's **own** HP, never below 0
	`heal(healing)` – raises HP, never above `hp_max`
	`attack(target)` – rolls the weapon's damage if one is equipped, otherwise uses `damage`, and calls `target.take_damage(...)`
	`introduce()`, `__repr__()`, `__str__()`

`attack()` never touches the target's `_hp` directly. It asks the target to take damage, so each object only changes its own data (encapsulation).

In [6]:
```
hero = Character("Hero", hp_max=50)
print(hero)
print(repr(hero))
hero.introduce()
```

```
Character: Hero with 50 HP and 10 damage
<Character name=Hero hp=50 damage=10>
Hi, my name is Hero!
```

HP stays between 0 and `hp_max`, and `hp` cannot be set from outside:

In [7]:
```
hero.take_damage(30)
print("After 30 damage:  ", hero.hp)
hero.heal(100)
print("After healing 100:", hero.hp, "(capped at hp_max)")
hero.take_damage(999)
print("After 999 damage: ", hero.hp, "| is_alive:", hero.is_alive)

try:
    hero.hp = 1000
except AttributeError as error:
    print("AttributeError:", error)
```

```
After 30 damage:   20
After healing 100: 50 (capped at hp_max)
After 999 damage:  0 | is_alive: False
AttributeError: property 'hp' of 'Character' object has no setter
```

Attacking with and without a weapon:

In [8]:
```
random.seed(3)
unarmed = Character("Unarmed")
armed = Character("Armed", weapon=Weapon("Sword", Dice(8)))
dummy = Character("Dummy", hp_max=1000)

unarmed.attack(dummy)
print("Unarmed hit -> dummy HP:", dummy.hp, "(fixed base damage 10)")
for _ in range(3):
    armed.attack(dummy)
    print("Sword hit   -> dummy HP:", dummy.hp)
```

```
Unarmed hit -> dummy HP: 990 (fixed base damage 10)
Sword hit   -> dummy HP: 986
Sword hit   -> dummy HP: 983
Sword hit   -> dummy HP: 977
```

`Warrior(Character)` (`character.py`)

A defensive specialist.

New attribute	`armor` (default 5)
Overridden methods	`take_damage(damage)` – reduces incoming damage by `armor` (never below 0) before calling `Character.take_damage()` via `super()`
	`introduce()`

Because `Character.attack()` calls `target.take_damage()`, the armor is applied automatically whenever a warrior is attacked, by any type of character.

In [9]:
```
warrior = Warrior("Conan", hp_max=50)
print(warrior, "| armor:", warrior.armor)
warrior.introduce()

warrior.take_damage(12)
print("12 damage -> HP:", warrior.hp, "(12 - 5 armor = 7)")
warrior.take_damage(3)
print(" 3 damage -> HP:", warrior.hp, "(3 - 5 armor = 0, no damage)")
```

```
Warrior: Conan with 50 HP and 10 damage | armor: 5
Hi, my name is Conan and I am a defensive specialist!
12 damage -> HP: 43 (12 - 5 armor = 7)
 3 damage -> HP: 43 (3 - 5 armor = 0, no damage)
```

`Mage(Character)` (`character.py`)

A resource-based attacker.

New attributes	`mana`, `mana_max` (both default 30)

Overridden methods	`attack(target)` : with at least 10 mana, spends 10 mana on a spell that deals **double base damage** and ignores any weapon. Otherwise it does a normal attack via `super().attack(target)` (using the weapon if one is equipped) and restores 5 mana (max `mana_max`).
	`introduce()`

In [10]:
```
mage = Mage("Merlin")
mage.introduce()
dummy = Character("Dummy", hp_max=200)

for turn in range(1, 7):
    mana_before = mage.mana
    mage.attack(dummy)
    kind = "spell " if mana_before >= 10 else "normal"
    print(f"Turn {turn}: {kind} attack | dummy HP {dummy.hp:3} | mana {mana_
```

```
Hi, my name is Merlin and I am a resource-based attacker!
Turn 1: spell  attack | dummy HP 180 | mana 30 -> 20
Turn 2: spell  attack | dummy HP 160 | mana 20 -> 10
Turn 3: spell  attack | dummy HP 140 | mana 10 ->  0
Turn 4: normal attack | dummy HP 130 | mana  0 ->  5
Turn 5: normal attack | dummy HP 120 | mana  5 -> 10
Turn 6: spell  attack | dummy HP 100 | mana 10 ->  0
```

The mage casts three spells (20 damage each), runs out of mana, and then does normal attacks that restore 5 mana until it can cast again.

The spell ignores the weapon, even when one is equipped:

In [11]:
```
random.seed(4)
armed_mage = Mage("Armed Merlin", mana=10, weapon=Weapon("Staff", Dice(8)))
dummy = Character("Dummy", hp_max=200)

armed_mage.attack(dummy)
print("Spell (mana 10)      -> dummy HP:", dummy.hp, "(always exactly 2 x 10
armed_mage.attack(dummy)
print("Normal attack (staff)-> dummy HP:", dummy.hp, "(d8 roll)")
```

```
Spell (mana 10)      -> dummy HP: 180 (always exactly 2 x 10)
Normal attack (staff)-> dummy HP: 176 (d8 roll)
```

`Battle` (`battle.py`)

Manages turn-based combat between two characters.

Attributes	`hero` , `enemy` , `max_rounds` (default 1000), `rounds_played`
Methods	`play_round()` – the hero attacks the enemy, and the enemy strikes back if it is still alive

	`simulate_battle(verbose=True)` – runs rounds in a `while` loop until one character is dead, and returns the winner (or `None` if `max_rounds` is reached)
	`__repr__()`

In [12]:
```
battle = Battle(Character("A"), Character("B"))
battle.play_round()
print(battle)
```

```
Battle(hero=<Character name=A hp=90 damage=10>, enemy=<Character name=B hp=9
0 damage=10>, rounds_played=1)
```

Part 2: Simulated battles

Battle 1: Warrior vs. Mage, without weapons

The warrior's armor reduces the mage's spells from 20 to 15 damage, and its normal attacks from 10 to 5.

In [13]:
```
random.seed(1)
winner = Battle(Warrior("Conan"), Mage("Merlin")).simulate_battle()
```

```
Round 1: Conan 85 HP, Merlin 90 HP
Round 2: Conan 70 HP, Merlin 80 HP
Round 3: Conan 55 HP, Merlin 70 HP
Round 4: Conan 50 HP, Merlin 60 HP
Round 5: Conan 45 HP, Merlin 50 HP
Round 6: Conan 30 HP, Merlin 40 HP
Round 7: Conan 25 HP, Merlin 30 HP
Round 8: Conan 20 HP, Merlin 20 HP
Round 9: Conan 5 HP, Merlin 10 HP
Round 10: Conan 5 HP, Merlin 0 HP
Conan wins after 10 rounds!
```

Battle 2: Warrior vs. Mage, with weapons

The warrior uses a greataxe (d12). The mage carries a staff (d8), which it only uses when it is out of mana.

In [14]:
```
random.seed(2)
conan = Warrior("Conan", weapon=Weapon("Greataxe", Dice(12)))
merlin = Mage("Merlin", weapon=Weapon("Staff", Dice(8)))
winner = Battle(conan, merlin).simulate_battle()
```

```
Round 1: Conan 85 HP, Merlin 99 HP
Round 2: Conan 70 HP, Merlin 97 HP
Round 3: Conan 55 HP, Merlin 95 HP
Round 4: Conan 55 HP, Merlin 89 HP
Round 5: Conan 55 HP, Merlin 77 HP
Round 6: Conan 40 HP, Merlin 72 HP
Round 7: Conan 40 HP, Merlin 62 HP
Round 8: Conan 40 HP, Merlin 52 HP
Round 9: Conan 25 HP, Merlin 42 HP
Round 10: Conan 25 HP, Merlin 31 HP
Round 11: Conan 23 HP, Merlin 24 HP
Round 12: Conan 8 HP, Merlin 12 HP
Round 13: Conan 7 HP, Merlin 3 HP
Round 14: Conan 7 HP, Merlin 0 HP
Conan wins after 14 rounds!
```

Battle 3: Armed vs. unarmed character

A plain `Character` with a sword (d20, average 10.5) against one using the fixed base damage of 10.

In [15]:
```
random.seed(3)
winner = Battle(Character("Swordsman", weapon=Weapon("Sword", Dice(20))), Ch
```

```
Round 1: Swordsman 90 HP, Brawler 92 HP
Round 2: Swordsman 80 HP, Brawler 73 HP
Round 3: Swordsman 70 HP, Brawler 55 HP
Round 4: Swordsman 60 HP, Brawler 50 HP
Round 5: Swordsman 50 HP, Brawler 38 HP
Round 6: Swordsman 40 HP, Brawler 18 HP
Round 7: Swordsman 30 HP, Brawler 2 HP
Round 8: Swordsman 30 HP, Brawler 0 HP
Swordsman wins after 8 rounds!
```

Battle 4: Nobody can deal damage

Two warriors with armor 10 and base damage 10 deal 0 damage to each other. Without a round limit, the `while` loop would never end. `max_rounds` stops the battle and `simulate_battle()` returns `None`.

In [16]:
```
tank_a = Warrior("Tank A", armor=10)
tank_b = Warrior("Tank B", armor=10)
winner = Battle(tank_a, tank_b, max_rounds=5).simulate_battle()
print("Returned winner:", winner)
```

```
Round 1: Tank A 100 HP, Tank B 100 HP
Round 2: Tank A 100 HP, Tank B 100 HP
Round 3: Tank A 100 HP, Tank B 100 HP
Round 4: Tank A 100 HP, Tank B 100 HP
Round 5: Tank A 100 HP, Tank B 100 HP
No winner after 5 rounds.
Returned winner: None
```

Many battles: who wins on average?

A single battle with dice is random, so we run 1000 battles per match-up and count the wins.

New characters are created for every battle (with a `lambda`), since a character keeps its lost HP after a battle.

In [17]:
```
def win_rates(make_hero, make_enemy, n=1000):
    """Run n battles with fresh characters and return the share of hero wins
    counts = {"hero": 0, "enemy": 0, "draw": 0}
    for _ in range(n):
        hero, enemy = make_hero(), make_enemy()
        winner = Battle(hero, enemy).simulate_battle(verbose=False)
        if winner is None:
            counts["draw"] += 1
        elif winner is hero:
            counts["hero"] += 1
        else:
            counts["enemy"] += 1
    return {key: value / n for key, value in counts.items()}

axe = lambda: Weapon("Greataxe", Dice(12))
staff = lambda: Weapon("Staff", Dice(8))

matchups = {
    "Warrior vs Mage (no weapons)":   (lambda: Warrior("W"), lambda: Mage("M
    "Mage vs Warrior (no weapons)":   (lambda: Mage("M"), lambda: Warrior("W
    "Warrior+axe vs Mage+staff":      (lambda: Warrior("W", weapon=axe()), l
    "Mage+staff vs Warrior+axe":      (lambda: Mage("M", weapon=staff()), la
    "Sword (d20) vs unarmed":         (lambda: Character("A", weapon=Weapon(
}

random.seed(5)
print(f"{'Match-up (hero first)':32} {'hero':>6} {'enemy':>6} {'draw':>6}")
for name, (make_hero, make_enemy) in matchups.items():
    r = win_rates(make_hero, make_enemy)
    print(f"{name:32} {r['hero']:6.1%} {r['enemy']:6.1%} {r['draw']:6.1%}")
```

```
Match-up (hero first)              hero  enemy   draw
Warrior vs Mage (no weapons)     100.0%   0.0%   0.0%
Mage vs Warrior (no weapons)     100.0%   0.0%   0.0%
Warrior+axe vs Mage+staff         41.5%  58.5%   0.0%
Mage+staff vs Warrior+axe         76.9%  23.1%   0.0%
Sword (d20) vs unarmed            65.2%  34.8%   0.0%
```